10.5_Encontrar_Mustra

April 27, 2026

[1]:
```
from pathlib import Path
import pandas as pd
import numpy as np
import re
import time
import json

from sklearn.model_selection import train_test_split
```

[2]:
```
PROJECT_ROOT = Path("/home/harielpadillasanchez/Documentos/TT/TT2")
DATA_DIR = PROJECT_ROOT / "data"
SPLIT_DIR = DATA_DIR / "splits"
SPLIT_DIR.mkdir(parents=True, exist_ok=True)

FEINA_PATH = DATA_DIR / "Dataset_FEINA.xlsx"
MUESTRA_PATH = DATA_DIR / "Muestra_csv.csv"
CLEAN_OUT_PATH = DATA_DIR / "Dataset_FEINA_clean_base_repr.csv"

PREFIX = "feina_repr30"
RANDOM_STATE = 42
SAMPLE_FRAC = 0.30

TRAIN_SIZE = 0.70
VAL_SIZE = 0.15
TEST_SIZE = 0.15
assert abs(TRAIN_SIZE + VAL_SIZE + TEST_SIZE - 1.0) < 1e-9
```

[3]:
```
t0 = time.perf_counter()

df_feina = pd.read_excel(FEINA_PATH)
df_muestra = pd.read_csv(MUESTRA_PATH)

t1 = time.perf_counter()

print(f"Tiempo de carga: {t1 - t0:.2f} segundos")
print("Shape FEINA original:", df_feina.shape)
print("Shape muestra manual:", df_muestra.shape)
print("\nColumnas FEINA:")
```

```
print(df_feina.columns.tolist())
```

```
Tiempo de carga: 0.41 segundos
Shape FEINA original: (5313, 15)
Shape muestra manual: (20, 17)

Columnas FEINA:
['Unnamed: 0', 'idFinal', 'Segmento', 'Propuesta', 'idcod', 'atr0', 'atr1',
'atr2', 'atr3', 'atr4', 'atr5', 'atr6', 'atr7', 'atr8', 'lex']
```

[4]:
```
df = df_feina.copy()

rename_map = {
    "Unnamed: 0": "row_id",
    "Segmento": "source_text",
    "Propuesta": "reference_text",
}

df = df.rename(columns=rename_map)

required_cols = ["row_id", "idFinal", "source_text", "reference_text"]
missing = [c for c in required_cols if c not in df.columns]

if missing:
    raise ValueError(f"Faltan columnas requeridas en FEINA: {missing}")

print("Shape después de renombrar:", df.shape)
display(df[required_cols].head(3))
```

```
Shape después de renombrar: (5313, 15)
```

	row_id	idFinal	source_text	reference_text
0	0	58_LibroBAC.pdf	Como antes explicamos, las finanzas y los conc…	Como antes explicamos, las finanzas están sust…
1	1	60_LibroBAC.pdf	Una vez dirimidos estos asuntos, se entra de l…	Una vez resueltos estos asuntos, se abordan al…
2	2	62_LibroBAC.pdf	Pero aquí no termina la utilidad del libro, ya…	Pero aquí no termina la utilidad del libro, pu…

[5]:
```
calibracion_inicial_ids = {3690, 1948, 286, 467, 2523}
few_shot_ids = {78, 1805, 3635, 5262}
muestra_ids = set(df_muestra["id"].dropna().astype(int).tolist())

exclude_ids = set()
exclude_ids.update(calibracion_inicial_ids)
exclude_ids.update(few_shot_ids)
exclude_ids.update(muestra_ids)
```

```
print("Total ids excluidos:", len(exclude_ids))
print("Ejemplo ids excluidos:", sorted(list(exclude_ids))[:10])
```

```
Total ids excluidos: 27
Ejemplo ids excluidos: [78, 224, 286, 329, 467, 507, 1756, 1765, 1805, 1948]
```

[6]:
```
df_clean = df[~df["row_id"].astype(int).isin(exclude_ids)].copy()
df_clean = df_clean.reset_index(drop=True)

print("Shape dataset limpio representativo base:", df_clean.shape)
print("IDs únicos:", df_clean["row_id"].nunique())

if df_clean["row_id"].nunique() != len(df_clean):
    print("Ojo: hay ids repetidos")
else:
    print("OK: un id por fila")

df_clean.to_csv(CLEAN_OUT_PATH, index=False, encoding="utf-8-sig")
print("Archivo limpio guardado en:", CLEAN_OUT_PATH)
```

```
Shape dataset limpio representativo base: (5286, 15)
IDs únicos: 5286
OK: un id por fila
Archivo limpio guardado en: /home/harielpadillasanchez/Documentos/TT/TT2/data/Da
taset_FEINA_clean_base_repr.csv
```

[7]:
```
RULES_MAP = {
    1: "Word Frequency",
    2: "Abstract vs. Concrete Language",
    3: "Visual Reference Dependence",
    4: "Ambiguity in Word Meaning",
    5: "Unnecessary Words",
    6: "Complex Phrases",
    7: "Use of Nominalizations",
    8: "Conjunctions",
    9: "Subordinating Conjunctions",
    10: "Past anterior and future perfect tense",
    11: "Subjunctive Mood Verb Forms",
    12: "Use of Gerund",
    13: "Abbreviations, Shortenings, Acronyms, and Initialisms",
    14: "Use of Numerical Expressions",
    15: "Sentence Length",
    16: "Parenthetical Phrases",
    17: "Relative Clauses",
    18: "Sentences of formal and aesthetic use",
    19: "Sentence Order",
    20: "Common Discourse Markers",
```

```
    21: "Use of Complex Punctuation Marks",
}

display(pd.DataFrame(
    [{"rule_id": k, "rule_name": v} for k, v in RULES_MAP.items()]
).sort_values("rule_id").reset_index(drop=True))
```

	rule_id	rule_name
0	1	Word Frequency
1	2	Abstract vs. Concrete Language
2	3	Visual Reference Dependence
3	4	Ambiguity in Word Meaning
4	5	Unnecessary Words
5	6	Complex Phrases
6	7	Use of Nominalizations
7	8	Conjunctions
8	9	Subordinating Conjunctions
9	10	Past anterior and future perfect tense
10	11	Subjunctive Mood Verb Forms
11	12	Use of Gerund
12	13	Abbreviations, Shortenings, Acronyms, and Init…
13	14	Use of Numerical Expressions
14	15	Sentence Length
15	16	Parenthetical Phrases
16	17	Relative Clauses
17	18	Sentences of formal and aesthetic use
18	19	Sentence Order
19	20	Common Discourse Markers
20	21	Use of Complex Punctuation Marks

[8]:
```
RULE_FAMILY_MAP = {
    1: "lexical",
    2: "lexical",
    3: "reduction",
    4: "lexical",
    5: "reduction",
    6: "lexical",
    7: "morphosyntactic",
    8: "morphosyntactic",
    9: "morphosyntactic",
    10: "morphosyntactic",
    11: "morphosyntactic",
    12: "morphosyntactic",
    13: "lexical",
    14: "structural",
    15: "structural",
    16: "reduction",
```

```
    17: "structural",
    18: "reduction",
    19: "structural",
    20: "lexical",
    21: "structural",
}

display(pd.DataFrame(
    [{"rule_id": k, "family": v, "rule_name": RULES_MAP.get(k, "UNKNOWN")} for␣
 ↪k, v in RULE_FAMILY_MAP.items()]
).sort_values("rule_id").reset_index(drop=True))
```

```
    rule_id           family  \
0         1          lexical
1         2          lexical
2         3        reduction
3         4          lexical
4         5        reduction
5         6          lexical
6         7  morphosyntactic
7         8  morphosyntactic
8         9  morphosyntactic
9        10  morphosyntactic
10       11  morphosyntactic
11       12  morphosyntactic
12       13          lexical
13       14       structural
14       15       structural
15       16        reduction
16       17       structural
17       18        reduction
18       19       structural
19       20          lexical
20       21       structural

                                        rule_name
0                                  Word Frequency
1                    Abstract vs. Concrete Language
2                       Visual Reference Dependence
3                         Ambiguity in Word Meaning
4                                 Unnecessary Words
5                                   Complex Phrases
6                              Use of Nominalizations
7                                      Conjunctions
8                           Subordinating Conjunctions
9              Past anterior and future perfect tense
10                        Subjunctive Mood Verb Forms
11                                     Use of Gerund
```

```
12  Abbreviations, Shortenings, Acronyms, and Init…
13                        Use of Numerical Expressions
14                                     Sentence Length
15                                Parenthetical Phrases
16                                     Relative Clauses
17              Sentences of formal and aesthetic use
18                                     Sentence Order
19                               Common Discourse Markers
20                    Use of Complex Punctuation Marks
```

[9]:
```python
RULE_COLS = [f"atr{i}" for i in range(9) if f"atr{i}" in df_clean.columns]
print("Columnas de reglas encontradas:", RULE_COLS)
```

```
Columnas de reglas encontradas: ['atr0', 'atr1', 'atr2', 'atr3', 'atr4', 'atr5',
'atr6', 'atr7', 'atr8']
```

[10]:
```python
def safe_rule_to_int(value):
    """
    Convierte un valor de regla a entero si es válido.
    Si es NaN o inválido, regresa None.
    """
    if pd.isna(value):
        return None
    try:
        return int(value)
    except Exception:
        return None


def extract_rules_from_row(row, rule_cols):
    """
    Extrae la lista ordenada de reglas presentes en una fila.
    """
    rules = []
    for col in rule_cols:
        rule_id = safe_rule_to_int(row.get(col))
        if rule_id is not None:
            rules.append(rule_id)
    return rules


def map_rules_to_families(rule_list, family_map):
    """
    Convierte una lista de rule_ids a familias, preservando orden
    y eliminando duplicados repetidos.
    """
    families = []
    seen = set()
```

```
    for rule_id in rule_list:
        fam = family_map.get(rule_id, "unknown")
        if fam not in seen:
            families.append(fam)
            seen.add(fam)
    return families
```

[11]:
```
df_feat = df_clean.copy()

df_feat["rules_list"] = df_feat.apply(lambda row: extract_rules_from_row(row,␣
 ↪RULE_COLS), axis=1)
df_feat["n_rules"] = df_feat["rules_list"].apply(len)
df_feat["main_rule"] = df_feat["rules_list"].apply(lambda x: x[0] if len(x) > 0␣
 ↪else np.nan)
df_feat["main_rule_name"] = df_feat["main_rule"].apply(
    lambda x: RULES_MAP.get(int(x), "UNKNOWN") if pd.notna(x) else np.nan
)

df_feat["families_list"] = df_feat["rules_list"].apply(lambda x:␣
 ↪map_rules_to_families(x, RULE_FAMILY_MAP))
df_feat["n_families"] = df_feat["families_list"].apply(len)
df_feat["main_family"] = df_feat["families_list"].apply(lambda x: x[0] if␣
 ↪len(x) > 0 else "unknown")

display(df_feat[[
    "row_id", "rules_list", "n_rules", "main_rule", "main_rule_name",
    "families_list", "n_families", "main_family"
]].head(10))
```

	row_id	rules_list	n_rules	main_rule	main_rule_name	families_list	n_families	main_family
0	0	[15, 21, 8, 5]	4	15.0	Sentence Length	[structural, morphosyntactic, reduction]	3	structural
1	1	[1, 5, 15, 17, 2, 19]	6	1.0	Word Frequency	[lexical, reduction, structural]	3	lexical
2	2	[15, 19, 5, 16, 6]	5	15.0	Sentence Length	[structural, reduction, lexical]	3	structural
3	3	[5, 15, 6]	3	5.0	Unnecessary Words	[reduction, structural, lexical]	3	reduction
4	4	[15, 12, 6, 5]	4	15.0	Sentence Length	[structural, morphosyntactic, lexical, reduction]	4	structural
5	5	[19, 5, 6, 1]	4	19.0	Sentence Order			
6	6	[15, 19, 5]	3	15.0	Sentence Length			
7	7	[19, 6]	2	19.0	Sentence Order			
8	8	[15, 19, 4, 5]	4	15.0	Sentence Length			
9	9	[15, 5, 11, 16]	4	15.0	Sentence Length			

```
5                    [structural, reduction, lexical]           3  structural
6                             [structural, reduction]           2  structural
7                               [structural, lexical]           2  structural
8                    [structural, lexical, reduction]           3  structural
9            [structural, reduction, morphosyntactic]           3  structural
```

[12]:
```python
print("Distribución de n_rules:")
display(df_feat["n_rules"].value_counts(dropna=False).sort_index())

print("\nDistribución de main_family:")
display(df_feat["main_family"].value_counts(dropna=False))

print("\nTop main_rule:")
display(
    df_feat["main_rule"]
    .value_counts(dropna=False)
    .rename_axis("main_rule")
    .reset_index(name="count")
    .head(15)
)
```

```
Distribución de n_rules:
```

```
n_rules
0      30
1    1439
2    1668
3    1171
4     598
5     254
6      91
7      27
8       8
Name: count, dtype: int64
```

```
Distribución de main_family:
```

```
main_family
reduction          1976
structural         1876
lexical            1135
morphosyntactic     269
unknown              30
Name: count, dtype: int64
```

```
Top main_rule:
```

```
    main_rule  count
0         5.0    980
```

```
1        15.0    888
2         6.0    532
3        16.0    496
4        21.0    342
5        18.0    329
6         1.0    273
7        19.0    243
8        14.0    206
9        17.0    197
10        3.0    171
11        4.0    127
12       12.0    107
13        2.0     87
14       13.0     87
```

[13]:
```python
def clean_text_for_stats(x):
    if pd.isna(x):
        return ""
    return str(x).strip()

def count_words(text):
    text = clean_text_for_stats(text)
    if not text:
        return 0
    return len(text.split())

def count_sentences(text):
    text = clean_text_for_stats(text)
    if not text:
        return 0
    parts = re.split(r"[.!?]+", text)
    parts = [p.strip() for p in parts if p.strip()]
    return len(parts)

def has_number(text):
    text = clean_text_for_stats(text)
    return int(bool(re.search(r"\d", text)))

def has_money(text):
    text = clean_text_for_stats(text)
    return int(bool(re.search(r"[$€£¥]|peso|pesos|dólar|dolares|dólares", text,␣
 ↪flags=re.IGNORECASE)))

def has_percent(text):
    text = clean_text_for_stats(text)
    return int(bool(re.search(r"%|por ciento", text, flags=re.IGNORECASE)))
```

```
df_feat["src_words"] = df_feat["source_text"].apply(count_words)
df_feat["src_sentences"] = df_feat["source_text"].apply(count_sentences)
df_feat["has_number"] = df_feat["source_text"].apply(has_number)
df_feat["has_money"] = df_feat["source_text"].apply(has_money)
df_feat["has_percent"] = df_feat["source_text"].apply(has_percent)

display(df_feat[[
    "row_id", "src_words", "src_sentences",
    "has_number", "has_money", "has_percent",
    "n_rules", "main_family", "lex"
]].head(10))
```

	row_id	src_words	src_sentences	has_number	has_money	has_percent	\
0	0	67	1	0	0	0	
1	1	58	1	0	0	0	
2	2	65	1	0	0	0	
3	3	51	1	0	0	0	
4	4	51	1	0	0	0	
5	5	29	1	0	0	0	
6	6	42	1	0	0	0	
7	7	24	1	0	0	0	
8	8	59	1	0	0	0	
9	9	72	1	0	0	0	

	n_rules	main_family	lex
0	4	structural	0
1	6	lexical	1
2	5	structural	0
3	3	reduction	0
4	4	structural	0
5	4	structural	1
6	3	structural	0
7	2	structural	0
8	4	structural	0
9	4	structural	0

[14]:
```
def bucket_n_rules(n):
    if pd.isna(n):
        return "unknown"
    n = int(n)
    if n <= 1:
        return "r1"
    elif n == 2:
        return "r2"
    elif n == 3:
        return "r3"
    else:
```

```
        return "r4plus"

def bucket_length(n_words):
    if pd.isna(n_words):
        return "unknown"
    n_words = int(n_words)
    if n_words <= 12:
        return "short"
    elif n_words <= 24:
        return "medium"
    else:
        return "long"

df_feat["n_rules_bucket"] = df_feat["n_rules"].apply(bucket_n_rules)
df_feat["length_bucket"] = df_feat["src_words"].apply(bucket_length)
df_feat["lex_bucket"] = df_feat["lex"].fillna(-1).astype(int).astype(str)

display(df_feat[[
    "row_id", "src_words", "length_bucket",
    "n_rules", "n_rules_bucket",
    "main_family", "has_number", "lex", "lex_bucket"
]].head(10))
```

```
   row_id  src_words length_bucket  n_rules n_rules_bucket main_family  \
0       0         67          long        4         r4plus  structural
1       1         58          long        6         r4plus     lexical
2       2         65          long        5         r4plus  structural
3       3         51          long        3             r3   reduction
4       4         51          long        4         r4plus  structural
5       5         29          long        4         r4plus  structural
6       6         42          long        3             r3  structural
7       7         24        medium        2             r2  structural
8       8         59          long        4         r4plus  structural
9       9         72          long        4         r4plus  structural

   has_number  lex lex_bucket
0           0    0          0
1           0    1          1
2           0    0          0
3           0    0          0
4           0    0          0
5           0    1          1
6           0    0          0
7           0    0          0
8           0    0          0
9           0    0          0
```

[15]:
```
df_feat["stratum"] = df_feat.apply(
    lambda r: " | ".join([
        f"fam={r['main_family']}",
        f"rules={r['n_rules_bucket']}",
        f"len={r['length_bucket']}",
        f"num={int(r['has_number'])}",
        f"lex={r['lex_bucket']}",
    ]),
    axis=1
)

print("Número de estratos finos:", df_feat["stratum"].nunique())
display(
    df_feat["stratum"]
    .value_counts()
    .rename_axis("stratum")
    .reset_index(name="count")
    .head(20)
)
```

```
Número de estratos finos: 147
```

	stratum	count
0	fam=reduction | rules=r1 | len=medium | num=0 …	364
1	fam=reduction | rules=r2 | len=medium | num=0 …	271
2	fam=structural | rules=r4plus | len=long | num…	256
3	fam=reduction | rules=r2 | len=long | num=0 | …	231
4	fam=structural | rules=r2 | len=long | num=0 |…	228
5	fam=structural | rules=r3 | len=long | num=0 |…	225
6	fam=reduction | rules=r3 | len=long | num=0 | …	185
7	fam=reduction | rules=r1 | len=long | num=0 | …	178
8	fam=lexical | rules=r2 | len=medium | num=0 | …	170
9	fam=reduction | rules=r4plus | len=long | num=…	152
10	fam=structural | rules=r2 | len=medium | num=0…	141
11	fam=reduction | rules=r1 | len=short | num=0 |…	131
12	fam=lexical | rules=r1 | len=medium | num=0 | …	124
13	fam=structural | rules=r4plus | len=long | num…	123
14	fam=structural | rules=r1 | len=long | num=0 |…	114
15	fam=lexical | rules=r1 | len=short | num=0 | l…	105
16	fam=structural | rules=r1 | len=medium | num=0…	98
17	fam=lexical | rules=r2 | len=medium | num=0 | …	84
18	fam=structural | rules=r1 | len=short | num=0 …	76
19	fam=structural | rules=r3 | len=medium | num=0…	74

[22]:
```
TARGET_N = round(len(df_feat) * SAMPLE_FRAC)
print("Tamaño objetivo del subset representativo:", TARGET_N)
print("30% de", len(df_feat), "=", TARGET_N)
```

```
Tamaño objetivo del subset representativo: 1586
```

30% de 5286 = 1586

[23]:
```python
def representative_sample_fixed_total(
    df_input: pd.DataFrame,
    stratum_col: str,
    target_n: int,
    random_state: int = 42,
):
    rng = np.random.RandomState(random_state)

    counts = (
        df_input[stratum_col]
        .value_counts(dropna=False)
        .rename_axis(stratum_col)
        .reset_index(name="n_group")
    )

    counts["raw_quota"] = counts["n_group"] / len(df_input) * target_n
    counts["base_take"] = np.floor(counts["raw_quota"]).astype(int)

    # Garantizar mínimo 1 por estrato si el grupo existe
    counts["base_take"] = counts.apply(
        lambda r: 1 if r["n_group"] > 0 and r["base_take"] == 0 else␣
↪r["base_take"],
        axis=1
    )

    # Nunca tomar más de lo disponible
    counts["base_take"] = counts[["base_take", "n_group"]].min(axis=1)

    current_total = int(counts["base_take"].sum())
    diff = target_n - current_total

    counts["fractional"] = counts["raw_quota"] - np.floor(counts["raw_quota"])

    # Si faltan ejemplos, asignar a los grupos con mayor parte decimal y aún␣
↪capacidad
    if diff > 0:
        candidates = counts[counts["base_take"] < counts["n_group"]].copy()
        candidates = candidates.sort_values(
            by=["fractional", "n_group"],
            ascending=[False, False]
        ).reset_index(drop=True)

        i = 0
        while diff > 0 and len(candidates) > 0:
            idx = candidates.index[i % len(candidates)]
```

```
            real_idx = candidates.loc[idx].name
            group_name = candidates.loc[idx, stratum_col]

            mask = counts[stratum_col] == group_name
            if int(counts.loc[mask, "base_take"].iloc[0]) < int(counts.
↪loc[mask, "n_group"].iloc[0]):
                counts.loc[mask, "base_take"] += 1
                diff -= 1

            candidates = counts[counts["base_take"] < counts["n_group"]].copy()
            candidates = candidates.sort_values(
                by=["fractional", "n_group"],
                ascending=[False, False]
            ).reset_index(drop=True)

            if len(candidates) == 0:
                break
            i += 1

    elif diff < 0:
        diff = abs(diff)
        candidates = counts[counts["base_take"] > 1].copy()
        candidates = candidates.sort_values(
            by=["fractional", "n_group"],
            ascending=[True, True]
        ).reset_index(drop=True)

        i = 0
        while diff > 0 and len(candidates) > 0:
            idx = candidates.index[i % len(candidates)]
            group_name = candidates.loc[idx, stratum_col]

            mask = counts[stratum_col] == group_name
            if int(counts.loc[mask, "base_take"].iloc[0]) > 1:
                counts.loc[mask, "base_take"] -= 1
                diff -= 1

            candidates = counts[counts["base_take"] > 1].copy()
            candidates = candidates.sort_values(
                by=["fractional", "n_group"],
                ascending=[True, True]
            ).reset_index(drop=True)

            if len(candidates) == 0:
                break
            i += 1
```

```
    sampled_parts = []
    take_map = dict(zip(counts[stratum_col], counts["base_take"]))

    for group_name, group_df in df_input.groupby(stratum_col, dropna=False):
        n_take = int(take_map.get(group_name, 0))
        if n_take > 0:
            sampled_group = group_df.sample(
                n=n_take,
                random_state=random_state
            )
            sampled_parts.append(sampled_group)

    df_sampled = pd.concat(sampled_parts, axis=0).copy()
    df_sampled = df_sampled.sort_values("row_id").reset_index(drop=True)

    return df_sampled, counts
```

[24]:
```
df_repr30, quota_df = representative_sample_fixed_total(
    df_input=df_feat,
    stratum_col="stratum",
    target_n=TARGET_N,
    random_state=RANDOM_STATE,
)

print("Shape base limpia:", df_feat.shape)
print("Shape muestra representativa:", df_repr30.shape)
print("Porcentaje real:", len(df_repr30) / len(df_feat))

display(df_repr30[[
    "row_id", "main_family", "n_rules_bucket",
    "length_bucket", "has_number", "lex", "stratum"
]].head(15))

print("\nResumen de cuotas por estrato:")
display(quota_df.head(20))
```

```
Shape base limpia: (5286, 31)
Shape muestra representativa: (1586, 31)
Porcentaje real: 0.30003783579265986
```

	row_id	main_family	n_rules_bucket	length_bucket	has_number	lex	\
0	5	structural	r4plus	long	0	1	
1	7	structural	r2	medium	0	0	
2	10	morphosyntactic	r4plus	long	0	0	
3	13	structural	r4plus	long	0	0	
4	15	lexical	r2	long	0	0	
5	18	structural	r4plus	long	1	0	
6	30	structural	r3	long	1	0	

7	32	morphosyntactic	r2	long	0	0
8	33	reduction	r3	long	0	0
9	44	morphosyntactic	r3	long	0	0
10	46	structural	r4plus	long	0	0
11	50	structural	r3	long	0	0
12	51	reduction	r3	medium	0	0
13	52	lexical	r2	medium	0	0
14	57	structural	r2	medium	0	0

	stratum
0	fam=structural | rules=r4plus | len=long | num…
1	fam=structural | rules=r2 | len=medium | num=0…
2	fam=morphosyntactic | rules=r4plus | len=long …
3	fam=structural | rules=r4plus | len=long | num…
4	fam=lexical | rules=r2 | len=long | num=0 | lex=0
5	fam=structural | rules=r4plus | len=long | num…
6	fam=structural | rules=r3 | len=long | num=1 |…
7	fam=morphosyntactic | rules=r2 | len=long | nu…
8	fam=reduction | rules=r3 | len=long | num=0 | …
9	fam=morphosyntactic | rules=r3 | len=long | nu…
10	fam=structural | rules=r4plus | len=long | num…
11	fam=structural | rules=r3 | len=long | num=0 |…
12	fam=reduction | rules=r3 | len=medium | num=0 …
13	fam=lexical | rules=r2 | len=medium | num=0 | …
14	fam=structural | rules=r2 | len=medium | num=0…

Resumen de cuotas por estrato:

	stratum	n_group	raw_quota	\
0	fam=reduction | rules=r1 | len=medium | num=0 …	364	109.213772	
1	fam=reduction | rules=r2 | len=medium | num=0 …	271	81.310253	
2	fam=structural | rules=r4plus | len=long | num…	256	76.809686	
3	fam=reduction | rules=r2 | len=long | num=0 | …	231	69.308740	
4	fam=structural | rules=r2 | len=long | num=0 |…	228	68.408627	
5	fam=structural | rules=r3 | len=long | num=0 |…	225	67.508513	
6	fam=reduction | rules=r3 | len=long | num=0 | …	185	55.507000	
7	fam=reduction | rules=r1 | len=long | num=0 | …	178	53.406735	
8	fam=lexical | rules=r2 | len=medium | num=0 | …	170	51.006432	
9	fam=reduction | rules=r4plus | len=long | num=…	152	45.605751	
10	fam=structural | rules=r2 | len=medium | num=0…	141	42.305335	
11	fam=reduction | rules=r1 | len=short | num=0 |…	131	39.304956	
12	fam=lexical | rules=r1 | len=medium | num=0 | …	124	37.204692	
13	fam=structural | rules=r4plus | len=long | num…	123	36.904654	
14	fam=structural | rules=r1 | len=long | num=0 |…	114	34.204313	
15	fam=lexical | rules=r1 | len=short | num=0 | l…	105	31.503973	
16	fam=structural | rules=r1 | len=medium | num=0…	98	29.403708	
17	fam=lexical | rules=r2 | len=medium | num=0 | …	84	25.203178	
18	fam=structural | rules=r1 | len=short | num=0 …	76	22.802876	

```
19  fam=structural | rules=r3 | len=medium | num=0…       74   22.202800
```

```
    base_take  fractional
0         109    0.213772
1          81    0.310253
2          77    0.809686
3          69    0.308740
4          68    0.408627
5          67    0.508513
6          55    0.507000
7          53    0.406735
8          51    0.006432
9          45    0.605751
10         42    0.305335
11         39    0.304956
12         37    0.204692
13         37    0.904654
14         34    0.204313
15         31    0.503973
16         29    0.403708
17         25    0.203178
18         23    0.802876
19         22    0.202800
```

[25]:
```python
def compare_distribution(df_base, df_sample, col):
    base_dist = (
        df_base[col].value_counts(normalize=True, dropna=False)
        .rename("base")
    )
    sample_dist = (
        df_sample[col].value_counts(normalize=True, dropna=False)
        .rename("sample")
    )
    comp = pd.concat([base_dist, sample_dist], axis=1).fillna(0.0)
    comp["abs_diff"] = (comp["base"] - comp["sample"]).abs()
    return comp.reset_index().rename(columns={"index": col})

for col in ["main_family", "n_rules_bucket", "length_bucket", "has_number",␣
 ↪"lex"]:
    print(f"\nDistribución comparada: {col}")
    display(compare_distribution(df_feat, df_repr30, col))
```

```
Distribución comparada: main_family
```

```
       main_family      base    sample  abs_diff
0        reduction  0.373818  0.372005  0.001813
1       structural  0.354900  0.352459  0.002441
```

```
2          lexical  0.214718  0.214376  0.000342
3  morphosyntactic  0.050889  0.056116  0.005227
4          unknown  0.005675  0.005044  0.000631


Distribución comparada: n_rules_bucket

  n_rules_bucket      base    sample  abs_diff
0             r2  0.315551  0.315889  0.000339
1             r1  0.277904  0.276797  0.001107
2             r3  0.221529  0.221311  0.000217
3         r4plus  0.185017  0.186003  0.000985


Distribución comparada: length_bucket

  length_bucket      base    sample  abs_diff
0          long  0.482974  0.480454  0.002520
1        medium  0.390655  0.389660  0.000995
2         short  0.126372  0.129887  0.003515


Distribución comparada: has_number

   has_number      base    sample  abs_diff
0           0  0.895006  0.885876  0.009129
1           1  0.104994  0.114124  0.009129


Distribución comparada: lex

   lex      base    sample  abs_diff
0    0  0.817064  0.813997  0.003066
1    1  0.182936  0.186003  0.003066
```

[26]:
```python
df_repr30["split_stratum"] = df_repr30.apply(
    lambda r: " | ".join([
        f"fam={r['main_family']}",
        f"rules={r['n_rules_bucket']}",
        f"len={r['length_bucket']}",
    ]),
    axis=1
)

split_counts = df_repr30["split_stratum"].value_counts()

print("Número de split_strata:", df_repr30["split_stratum"].nunique())
print("Mínimo tamaño por split_stratum:", split_counts.min())

display(
    split_counts
```

```
    .rename_axis("split_stratum")
    .reset_index(name="count")
    .head(20)
)
```

```
Número de split_strata: 49
Mínimo tamaño por split_stratum: 1
```

	split_stratum	count
0	fam=structural | rules=r4plus | len=long	137
1	fam=reduction | rules=r1 | len=medium	111
2	fam=reduction | rules=r2 | len=medium	96
3	fam=structural | rules=r3 | len=long	95
4	fam=lexical | rules=r2 | len=medium	80
5	fam=structural | rules=r2 | len=long	79
6	fam=reduction | rules=r2 | len=long	78
7	fam=reduction | rules=r3 | len=long	73
8	fam=reduction | rules=r4plus | len=long	70
9	fam=structural | rules=r2 | len=medium	59
10	fam=reduction | rules=r1 | len=long	55
11	fam=lexical | rules=r1 | len=medium	46
12	fam=lexical | rules=r1 | len=short	43
13	fam=structural | rules=r3 | len=medium	41
14	fam=reduction | rules=r1 | len=short	40
15	fam=structural | rules=r1 | len=medium	40
16	fam=structural | rules=r1 | len=long	38
17	fam=lexical | rules=r3 | len=medium	38
18	fam=lexical | rules=r4plus | len=long	36
19	fam=reduction | rules=r3 | len=medium	36

[27]:
```
ID_COL = "row_id"

split_counts = df_repr30["split_stratum"].value_counts()
can_stratify_first_split = (
    df_repr30["split_stratum"].nunique() > 1
    and split_counts.min() >= 2
)

df_train, df_temp = train_test_split(
    df_repr30,
    test_size=(1 - TRAIN_SIZE),
    random_state=RANDOM_STATE,
    shuffle=True,
    stratify=df_repr30["split_stratum"] if can_stratify_first_split else None,
)

temp_counts = df_temp["split_stratum"].value_counts()
can_stratify_second_split = (
```

```
    df_temp["split_stratum"].nunique() > 1
    and temp_counts.min() >= 2
)

df_val, df_test = train_test_split(
    df_temp,
    test_size=0.50,
    random_state=RANDOM_STATE,
    shuffle=True,
    stratify=df_temp["split_stratum"] if can_stratify_second_split else None,
)

df_train = df_train.sort_values(by=ID_COL).reset_index(drop=True)
df_val = df_val.sort_values(by=ID_COL).reset_index(drop=True)
df_test = df_test.sort_values(by=ID_COL).reset_index(drop=True)

print("Train shape:", df_train.shape)
print("Val shape:", df_val.shape)
print("Test shape:", df_test.shape)

print("\nProporciones dentro del subset representativo:")
print("Train:", len(df_train) / len(df_repr30))
print("Val:", len(df_val) / len(df_repr30))
print("Test:", len(df_test) / len(df_repr30))

print("\n¿Se usó estratificación en split 1?", can_stratify_first_split)
print("¿Se usó estratificación en split 2?", can_stratify_second_split)
```

```
Train shape: (1110, 32)
Val shape: (238, 32)
Test shape: (238, 32)

Proporciones dentro del subset representativo:
Train: 0.699873896595208
Val: 0.15006305170239598
Test: 0.15006305170239598

¿Se usó estratificación en split 1? False
¿Se usó estratificación en split 2? False
```

[28]:
```
train_ids = set(df_train[ID_COL].astype(int).tolist())
val_ids = set(df_val[ID_COL].astype(int).tolist())
test_ids = set(df_test[ID_COL].astype(int).tolist())

overlap_train_val = train_ids & val_ids
overlap_train_test = train_ids & test_ids
overlap_val_test = val_ids & test_ids
```

```
print("Traslape train-val :", len(overlap_train_val))
print("Traslape train-test:", len(overlap_train_test))
print("Traslape val-test  :", len(overlap_val_test))

if len(overlap_train_val) == 0 and len(overlap_train_test) == 0 and␣
 ↪len(overlap_val_test) == 0:
    print("OK: no hay fuga entre splits.")
else:
    print("Ojo: sí hay traslapes.")
```

```
Traslape train-val : 0
Traslape train-test: 0
Traslape val-test  : 0
OK: no hay fuga entre splits.
```

[29]:
```
subset_path = SPLIT_DIR / f"{PREFIX}_subset.csv"
train_path = SPLIT_DIR / f"{PREFIX}_train.csv"
val_path = SPLIT_DIR / f"{PREFIX}_val.csv"
test_path = SPLIT_DIR / f"{PREFIX}_test.csv"
meta_path = SPLIT_DIR / f"{PREFIX}_metadata.json"

df_repr30.to_csv(subset_path, index=False, encoding="utf-8-sig")
df_train.to_csv(train_path, index=False, encoding="utf-8-sig")
df_val.to_csv(val_path, index=False, encoding="utf-8-sig")
df_test.to_csv(test_path, index=False, encoding="utf-8-sig")

metadata = {
    "prefix": PREFIX,
    "clean_base_path": str(CLEAN_OUT_PATH),
    "sample_frac": SAMPLE_FRAC,
    "target_n": TARGET_N,
    "random_state": RANDOM_STATE,
    "train_size": TRAIN_SIZE,
    "val_size": VAL_SIZE,
    "test_size": TEST_SIZE,
    "subset_shape": list(df_repr30.shape),
    "train_shape": list(df_train.shape),
    "val_shape": list(df_val.shape),
    "test_shape": list(df_test.shape),
    "overlap_train_val": len(overlap_train_val),
    "overlap_train_test": len(overlap_train_test),
    "overlap_val_test": len(overlap_val_test),
    "sampling_stratum_features": [
        "main_family",
        "n_rules_bucket",
        "length_bucket",
```

```
        "has_number",
        "lex",
    ],
    "split_stratum_features": [
        "main_family",
        "n_rules_bucket",
        "length_bucket",
    ],
}

with open(meta_path, "w", encoding="utf-8") as f:
    json.dump(metadata, f, ensure_ascii=False, indent=2)

print("Archivos guardados:")
print("-", subset_path)
print("-", train_path)
print("-", val_path)
print("-", test_path)
print("-", meta_path)
```

```
Archivos guardados:
-
/home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_repr30_subset.csv
-
/home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_repr30_train.csv
- /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_repr30_val.csv
- /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_repr30_test.csv
- /home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_repr30_metadata
.json
```